```
public int scoreForFrame(int theFrame)
{
  ball = 0;
  int score=0;
  for (int currentFrame = 0;
       currentFrame < theFrame;
       currentFrame++)
  {
    firstThrow = itsThrows[ball++];
    if (firstThrow == 10)
    {
      score += 10 + itsThrows[ball] + itsThrows[ball+1];
    }
    else
    {
      secondThrow = itsThrows[ball++];

      int frameScore = firstThrow + secondThrow;
      // spare needs next frames first throw
      if ( frameScore == 10 )
        score += frameScore + itsThrows[ball];
      else
        score += frameScore;
    }

  }
  return score;
}
private int ball;
private int firstThrow;
private int secondThrow;

private int itsScore = 0;
private int[] itsThrows = new int[21];
private int itsCurrentThrow = 0;
private int itsCurrentFrame = 1;
private boolean firstThrowInFrame = true;
```

RSK: I hadn't expected the name collision. We already had an instance variable named `firstThrow`. But it is better named `firstThrowInFrame`. Anyway, this works now. So we can pull the `else` clause out into its own function.

```
public int scoreForFrame(int theFrame)
{
  ball = 0;
  int score=0;
  for (int currentFrame = 0;
       currentFrame < theFrame;
       currentFrame++)
  {
    firstThrow = itsThrows[ball++];
    if (firstThrow == 10)
    {
      score += 10 + itsThrows[ball] + itsThrows[ball+1];
    }
```

```
    else
    {
      score += handleSecondThrow();
    }
  }

  return score;
}

private int handleSecondThrow()
{
  int score = 0;
  secondThrow = itsThrows[ball++];

  int frameScore = firstThrow + secondThrow;
  // spare needs next frames first throw
  if ( frameScore == 10 )
    score += frameScore + itsThrows[ball];
  else
    score += frameScore;
  return score;
}
```

RCM: Look at the structure of `scoreForFrame`! In pseudocode it looks something like this:

```
if strike
  score += 10 + nextTwoBalls();
else
  handleSecondThrow.
```

RCM: What if we changed it to

```
if strike
  score += 10 + nextTwoBalls();
else if spare
  score += 10 + nextBall();
else
  score += twoBallsInFrame()
```

RSK: Geez! That's pretty much the rules for scoring bowling isn't it? OK, let's see if we can get that structure in the real function. First, let's change the way the `ball` variable is being incremented, so that the three cases manipulate it independently.

```
public int scoreForFrame(int theFrame)
{
  ball = 0;
  int score=0;
  for (int currentFrame = 0;
       currentFrame < theFrame;
       currentFrame++)
  {
    firstThrow = itsThrows[ball];
    if (firstThrow == 10)
```

```
    {
      ball++;
      score += 10 + itsThrows[ball] + itsThrows[ball+1];
    }
    else
    {
      score += handleSecondThrow();
    }
  }

  return score;
}

private int handleSecondThrow()
{
  int score = 0;
  secondThrow = itsThrows[ball+1];

  int frameScore = firstThrow + secondThrow;
  // spare needs next frames first throw
  if ( frameScore == 10 )
  {
    ball+=2;
    score += frameScore + itsThrows[ball];
  }
  else
  {
    ball+=2;
    score += frameScore;
  }
  return score;
}
```

RCM: (grabs keyboard) OK, now let's get rid of the `firstThrow` and `secondThrow` variables and replace them with appropriate functions.

```
public int scoreForFrame(int theFrame)
{
  ball = 0;
  int score=0;
  for (int currentFrame = 0;
       currentFrame < theFrame;
       currentFrame++)
  {
    firstThrow = itsThrows[ball];
    if (strike())
    {
    ball++;
      score += 10 + nextTwoBalls();
    }
    else
    {
      score += handleSecondThrow();
    }
  }
```

```
    return score;
}

private boolean strike()
{
  return itsThrows[ball] == 10;
}
private int nextTwoBalls()
{
  return itsThrows[ball] + itsThrows[ball+1];
}
```

RCM: That step works, let's keep going.

```
private int handleSecondThrow()
{
  int score = 0;
  secondThrow = itsThrows[ball+1];

  int frameScore = firstThrow + secondThrow;
  // spare needs next frames first throw
  if ( spare() )
  {
    ball+=2;
    score += 10 + nextBall();
  }
  else
  {
    ball+=2;
    score += frameScore;
  }
  return score;
}

private boolean spare()
{
  return (itsThrows[ball] + itsThrows[ball+1]) == 10;
}

private int nextBall()
{
  return itsThrows[ball];
}
```

RCM: OK, that works too. Now let's deal with `frameScore`.

```
private int handleSecondThrow()
{
  int score = 0;
  secondThrow = itsThrows[ball+1];

  int frameScore = firstThrow + secondThrow;
  // spare needs next frames first throw
  if ( spare() )
```

```
    {
      ball+=2;
      score += 10 + nextBall();
    }
    else
    {
      score += twoBallsInFrame();
      ball+=2;
    }
    return score;
}

private int twoBallsInFrame()
{
  return itsThrows[ball] + itsThrows[ball+1];
}
```

RSK: Bob, you aren't incrementing `ball` in a consistent manner. In the spare and strike case, you increment before you calculate the score. In the `twoBallsInFrame` case you increment *after* you calculate the score. And the code *depends* on this order! What's up?

RCM: Sorry, I should have explained. I'm planning on moving the increments into `strike`, `spare`, and `twoBallsInFrame`. That way, they'll disappear from the `scoreForFrame` function, and the function will look just like our pseudocode.

RSK: OK, I'll trust you for a few more steps, but remember, I'm watching.

RCM: OK, now since nobody uses `firstThrow`, `secondThrow`, and `frameScore` anymore, we can get rid of them.

```
public int scoreForFrame(int theFrame)
{
  ball = 0;
  int score=0;
  for (int currentFrame = 0;
       currentFrame < theFrame;
       currentFrame++)
  {
    if (strike())
    {
      ball++;
      score += 10 + nextTwoBalls();
    }
    else
    {
      score += handleSecondThrow();
    }
  }

  return score;
}

private int handleSecondThrow()
{
  int score = 0;
  // spare needs next frames first throw
```

```
    if ( spare() )
    {
      ball+=2;
      score += 10 + nextBall();
    }
    else
    {
      score += twoBallsInFrame();
      ball+=2;
    }
    return score;
  }
```

RCM: (The sparkle in his eyes is a reflection of the green bar.) Now, since the only variable that couples the three cases is `ball`, and since `ball` is dealt with independently in each case, we can merge the three cases together.

```
public int scoreForFrame(int theFrame)
{
  ball = 0;
  int score=0;
  for (int currentFrame = 0;
       currentFrame < theFrame;
       currentFrame++)
  {
    if (strike())
    {
      ball++;
      score += 10 + nextTwoBalls();
    }
    else if ( spare() )
    {
      ball+=2;
      score += 10 + nextBall();
    }
    else
    {
      score += twoBallsInFrame();
      ball+=2;
    }
  }
  return score;
}
```

RSK: OK, now we can make the increments consistent and rename the functions to be more explicit. (grabs keyboard)

```
public int scoreForFrame(int theFrame)
{
  ball = 0;
  int score=0;
  for (int currentFrame = 0;
       currentFrame < theFrame;
       currentFrame++)
```

```
    {
      if (strike())
      {
        score += 10 + nextTwoBallsForStrike();
        ball++;
      }
      else if ( spare() )
      {
        score += 10 + nextBallForSpare();
        ball+=2;
      }
      else
      {
        score += twoBallsInFrame();
        ball+=2;
      }
    }

    return score;
  }

  private int nextTwoBallsForStrike()
  {
    return itsThrows[ball+1] + itsThrows[ball+2];
  }

  private int nextBallForSpare()
  {
    return itsThrows[ball+2];
  }
```

RCM: Look at that `scoreForFrame` function! That's the rules of bowling stated about as succinctly as possible.

RSK: But, Bob, what happened to the linked list of `Frame` objects? (snicker, snicker)

RCM: (sigh) We were bedevilled by the daemons of diagrammatic overdesign. My God, three little boxes drawn on the back of a napkin, `Game`, `Frame`, and `Throw`, and it was still too complicated and just plain wrong.

RSK: We made a mistake starting with the `Throw` class. We should have started with the `Game` class first!

RCM: Indeed! So, next time let's try starting at the highest level and work down.

RSK: (gasp) Top-down design!??!?!?

RCM: Correction, top-down, *test-first* design. Frankly, I don't know if this is a good rule or not. It's just what would have helped us in this case. So next time, I'm going to try it and see what happens.

RSK: Yeah, OK. Anyway, we still have some refactoring to do. The `ball` variable is just a private iterator for `scoreForFrame` and its minions. They should all be moved into a different object.

RCM: Oh, yes, your `Scorer` object. You were right after all. Let's do it.

RSK: (grabs keyboard and takes several small steps punctuated by tests to create . . .)

```
//Game.java---------------------------------
public class Game
```

```
{
  public int score()
  {
    return scoreForFrame(getCurrentFrame()-1);
  }

  public int getCurrentFrame()
  {
    return itsCurrentFrame;
  }

  public void add(int pins)
  {
    itsScorer.addThrow(pins);
    itsScore += pins;
    adjustCurrentFrame(pins);
  }

  private void adjustCurrentFrame(int pins)
  {
    if (firstThrowInFrame == true)
    {
    if( pins == 10 ) // strike
      itsCurrentFrame++;
    else
      firstThrowInFrame = false;
    }
    else
    {
    firstThrowInFrame=true;
    itsCurrentFrame++;
  }
    itsCurrentFrame = Math.min(11, itsCurrentFrame);
}

public int scoreForFrame(int theFrame)
{
  return itsScorer.scoreForFrame(theFrame);
}

private int itsScore = 0;
private int itsCurrentFrame = 1;
private boolean firstThrowInFrame = true;
private Scorer itsScorer = new Scorer();
}

//Scorer.java-----------------------------------
public class Scorer
{
  public void addThrow(int pins)
  {
    itsThrows[itsCurrentThrow++] = pins;
  }

  public int scoreForFrame(int theFrame)
```

```
  {
    ball = 0;
    int score=0;
    for (int currentFrame = 0;
         currentFrame < theFrame;
         currentFrame++)
    {
      if (strike())
      {
        score += 10 + nextTwoBallsForStrike();
          ball++;
      }
      else if ( spare() )
      {
        score += 10 + nextBallForSpare();
        ball+=2;
      }
      else
      {
        score += twoBallsInFrame();
        ball+=2;
      }
    }
    return score;
  }

  private boolean strike()
  {
    return itsThrows[ball] == 10;
  }

  private boolean spare()
  {
    return (itsThrows[ball] + itsThrows[ball+1]) == 10;
  }

  private int nextTwoBallsForStrike()
  {
    return itsThrows[ball+1] + itsThrows[ball+2];
  }

  private int nextBallForSpare()
  {
    return itsThrows[ball+2];
  }

  private int twoBallsInFrame()
  {
    return itsThrows[ball] + itsThrows[ball+1];
  }

  private int ball;
  private int[] itsThrows = new int[21];
  private int itsCurrentThrow = 0;
}
```

RSK: That's much better. Now, the `Game` just keeps track of frames, and the `Scorer` just calculates the score. The Single Responsibility Principle rocks!

RCM: Whatever. But it is better. Did you notice that the `itsScore` variable is not being used anymore?

RSK: Ha! You're right. Let's kill it. (gleefully starts erasing things)

```
public void add(int pins)
{
  itsScorer.addThrow(pins);
  adjustCurrentFrame(pins);
}
```

RSK: Not bad. Now, should we clean up the `adjustCurrentFrame` stuff?

RCM: OK, let's look at it.

```
private void adjustCurrentFrame(int pins)
{
  if (firstThrowInFrame == true)
  {
    if( pins == 10 ) // strike
      itsCurrentFrame++;
    else
      firstThrowInFrame = false;
  }
  else
  {
    firstThrowInFrame=true;
    itsCurrentFrame++;
  }
  itsCurrentFrame = Math.min(11, itsCurrentFrame);
}
```

RCM: OK, first let's extract the increments into a single function that also restricts the frame to 11. (Brrrr. I still don't like that 11.)

RSK: Bob, 11 means end of game.

RCM: Yeah. Brrrr. (grabs keyboard, makes a couple of changes punctuated by tests)

```
private void adjustCurrentFrame(int pins)
{
  if (firstThrowInFrame == true)
  {
    if( pins == 10 ) // strike
      advanceFrame();
    else
      firstThrowInFrame = false;
  }
  else
  {
    firstThrowInFrame=true;
    advanceFrame();
  }
}
```

```
private void advanceFrame()
{
  itsCurrentFrame = Math.min(11, itsCurrentFrame + 1);
}
```

RCM: OK, that's a little better. Now let's break out the strike case into its own function. (Takes a few small steps and runs tests between each.)

```
private void adjustCurrentFrame(int pins)
{
  if (firstThrowInFrame == true)
  {
    if (adjustFrameForStrike(pins) == false)
      firstThrowInFrame = false;
  }
  else
  {
    firstThrowInFrame=true;
    advanceFrame();
  }
}

private boolean adjustFrameForStrike(int pins)
{
  if (pins == 10)
  {
    advanceFrame();
    return true;
  }
  return false;
}
```

RCM: That's pretty good. Now, about that 11.

RSK: You really hate that don't you.

RCM: Yeah, look at the `score()` function,

```
public int score()
{
  return scoreForFrame(getCurrentFrame()-1);
}
```

RCM: That `-1` is odd. It's the only place we truly use `getCurrentFrame`, and yet we need to adjust what it returns.

RSK: Damn, you're right. How many times have we reversed ourselves on this?

RCM: Too many. But there it is. The code wants `itsCurrentFrame` to represent the frame of the last thrown ball, not the frame we are about to throw into.

RSK: Sheesh, that's going to break lots of test cases.

RCM: Actually, I think we should remove `getCurrentFrame` from all the test cases, and remove the `getCurrentFrame` function itself. Nobody really uses it.

RSK: OK, I get your point. I'll do it. It'll be like putting a lame horse out of its misery. (grabs keyboard)

```
//Game.java----------------------------------
  public int score()
  {
    return scoreForFrame(itsCurrentFrame);
  }
  private void advanceFrame()
  {
    itsCurrentFrame = Math.min(10, itsCurrentFrame + 1);
  }
```

RCM: Oh, for crying out loud. You mean to tell me that we were fretting over *that*. All we did was change the limit from 11 to 10 and remove the `-1`. Cripes!

RSK: Yeah, Uncle Bob, it really wasn't worth all the angst we gave it.

RCM: I hate the side effect in `adjustFrameForStrike()`. I want to get rid of it. What do you think of this?

```
private void adjustCurrentFrame(int pins)
{
    if ((firstThrowInFrame && pins == 10) ||
        (!firstThrowInFrame))
      advanceFrame();
    else
      firstThrowInFrame = false;
}
```

RSK: I like the idea, and it passes the tests, but I hate the long `if` statement. How about this?

```
private void adjustCurrentFrame(int pins)
{
  if (strike(pins) || !firstThrowInFrame)
    advanceFrame();
  else
    firstThrowInFrame = false;
}

private boolean strike(int pins)
{
  return (firstThrowInFrame && pins == 10);
}
```

RCM: Yeah, that's pretty. We could even go one step further.

```
private void adjustCurrentFrame(int pins)
{
  if (lastBallInFrame(pins))
    advanceFrame();
  else
    firstThrowInFrame = false;
}

private boolean lastBallInFrame(int pins)
{
  return strike(pins) || !firstThrowInFrame;
}
```

RSK: Nice!

RCM: OK, looks like we are done. Let's just read through the whole program and see if it's as simple and communicative as it can be.

```
//Game.java----------------------------------
public class Game
{
  public int score()
  {
    return scoreForFrame(itsCurrentFrame);
  }

public void add(int pins)
{
  itsScorer.addThrow(pins);
  adjustCurrentFrame(pins);
}

private void adjustCurrentFrame(int pins)
{
  if (lastBallInFrame(pins))
    advanceFrame();
  else
    firstThrowInFrame = false;
}

private boolean lastBallInFrame(int pins)
{
  return strike(pins) || !firstThrowInFrame;
}

private boolean strike(int pins)
{
  return (firstThrowInFrame && pins == 10);
}

private void advanceFrame()
{
  itsCurrentFrame = Math.min(10, itsCurrentFrame + 1);
}

public int scoreForFrame(int theFrame)
{
  return itsScorer.scoreForFrame(theFrame);
}

private int itsCurrentFrame = 0;
private boolean firstThrowInFrame = true;
private Scorer itsScorer = new Scorer();
}

//Scorer.java--------------------------------
public class Scorer
{
  public void addThrow(int pins)
```

```
    {
      itsThrows[itsCurrentThrow++] = pins;
    }

  public int scoreForFrame(int theFrame)
  {
    ball = 0;
    int score=0;
    for (int currentFrame = 0;
         currentFrame < theFrame;
         currentFrame++)
    {
      if (strike())
      {
        score += 10 + nextTwoBallsForStrike();
          ball++;
      }
      else if ( spare() )
      {
        score += 10 + nextBallForSpare();
        ball+=2;
      }
      else
      {
        score += twoBallsInFrame();
        ball+=2;
      }
    }
    return score;
  }

  private boolean strike()
  {
    return itsThrows[ball] == 10;
  }

  private boolean spare()
  {
    return (itsThrows[ball] + itsThrows[ball+1]) == 10;
  }

  private int nextTwoBallsForStrike()
  {
    return itsThrows[ball+1] + itsThrows[ball+2];
  }

  private int nextBallForSpare()
  {
    return itsThrows[ball+2];
  }

  private int twoBallsInFrame()
  {
    return itsThrows[ball] + itsThrows[ball+1];
  }
```

```
private int ball;
private int[] itsThrows = new int[21];
private int itsCurrentThrow = 0;
}
```

RCM: OK, that looks pretty good. I can't think of anything else to do.

RSK: Yeah, it's pretty. Let's look over the tests for good measure.

```
//TestGame.java-----------------------------------------
import junit.framework.*;

public class TestGame extends TestCase
{
  public TestGame(String name)
  {
    super(name);
  }

  private Game g;

  public void setUp()
  {
    g = new Game();
  }

  public void testTwoThrowsNoMark()
  {
    g.add(5);
    g.add(4);
    assertEquals(9, g.score());
  }

  public void testFourThrowsNoMark()
  {
    g.add(5);
    g.add(4);
    g.add(7);
    g.add(2);
    assertEquals(18, g.score());
    assertEquals(9,  g.scoreForFrame(1));
    assertEquals(18, g.scoreForFrame(2));
  }

  public void testSimpleSpare()
  {
    g.add(3);
    g.add(7);
    g.add(3);
    assertEquals(13, g.scoreForFrame(1));
  }

  public void testSimpleFrameAfterSpare()
  {
    g.add(3);
    g.add(7);
```

```
    g.add(3);
    g.add(2);
    assertEquals(13, g.scoreForFrame(1));
    assertEquals(18, g.scoreForFrame(2));
    assertEquals(18, g.score());
  }

  public void testSimpleStrike()
  {
    g.add(10);
    g.add(3);
    g.add(6);
    assertEquals(19, g.scoreForFrame(1));
    assertEquals(28, g.score());
  }

  public void testPerfectGame()
  {
    for (int i=0; i<12; i++)
    {
    g.add(10);
  }
  assertEquals(300, g.score());
}

public void testEndOfArray()
{
  for (int i=0; i<9; i++)
  {
    g.add(0);
    g.add(0);
  }
  g.add(2);
  g.add(8); // 10th frame spare
  g.add(10); // Strike in last position of array.
  assertEquals(20, g.score());
}

  public void testSampleGame()
  {
    g.add(1);
    g.add(4);
    g.add(4);
    g.add(5);
    g.add(6);
    g.add(4);
    g.add(5);
    g.add(5);
    g.add(10);
    g.add(0);
    g.add(1);
    g.add(7);
    g.add(3);
    g.add(6);
    g.add(4);
```

```
    g.add(10);
    g.add(2);
    g.add(8);
    g.add(6);
    assertEquals(133, g.score());
  }

  public void testHeartBreak()
  {
    for (int i=0; i<11; i++)
      g.add(10);
    g.add(9);
    assertEquals(299, g.score());
  }

  public void testTenthFrameSpare()
  {
    for (int i=0; i<9; i++)
      g.add(10);
    g.add(9);
    g.add(1);
    g.add(1);
    assertEquals(270, g.score());
  }
}
```

RSK: That pretty much covers it. Can you think of any more meaningful test cases?

RCM: No, I think that's the set. There aren't any there that I'd be comfortable removing at this point.

RSK: Then we're done.

RCM: I'd say so. Thanks a lot for your help.

RSK: No problem, it was fun.

Conclusion

After writing this chapter, I published it on the Object Mentor Web site.[3] Many people read it and gave their comments. Some folks were disturbed that there was almost no object-oriented design involved. I find this response interesting. Must we have object-oriented design in every application and every program? Here is a case where the program simply didn't need much of it. The `Scorer` class was really the only concession to OO, and even that was more simple partitioning than true OOD.

Other folks thought that there really should be a `Frame` class. One person went so far as to create a version of the program that contained a `Frame` class. It was much larger and more complex than what you see above.

Some folks felt that we weren't fair to UML. After all, we didn't do a complete design before we began. The funny little UML diagram on the back of the napkin (Figure 6-2) was not a complete design. It did not include sequence diagrams. I find this argument rather odd. It doesn't seem likely to me that adding sequence diagrams to Figure 6-2 would have caused us to abandon the `Throw` and `Frame` classes. Indeed, I think it would have entrenched us in our view that these classes were necessary.

Am I trying to say that diagrams are inappropriate? Of course not. Well, actually, yes, in a way I am. For *this* program, the diagrams didn't help at all. Indeed, they were a distraction. If we had followed them, we would have wound up with a program that was much more complex than necessary. You might contend that we would also

3. `http://www.objectmentor.com`

have wound up with a program that was more maintainable, but I disagree. The program we just went through is easy to understand and therefore easy to maintain. There are no mismanaged dependencies within it that make it rigid or fragile.

So, yes, diagrams can be inappropriate at times. When are they inappropriate? When you create them without code to validate them, *and then intend to follow them.* There is nothing wrong with drawing a diagram to explore an idea. However, having produced a diagram, you should not assume that it is the best design for the task. You may find that the best design will evolve as you take tiny little steps, writing tests first.

An Overview of the Rules of Bowling

Bowling is a game that is played by throwing a cantaloupe-sized ball down a narrow alley toward ten wooden pins. The object is to knock down as many pins as possible per throw.

The game is played in ten frames. At the beginning of each frame, all ten pins are set up. The player then gets two tries to knock them all down.

If the player knocks all the pins down on the first try, it is called a "strike," and the frame ends.

If the player fails to knock down all the pins with his first ball, but succeeds with the second ball, it is called a "spare."

After the second ball of the frame, the frame ends even if there are still pins standing.

A strike frame is scored by adding ten, plus the number of pins knocked down by the next two balls, to the score of the previous frame.

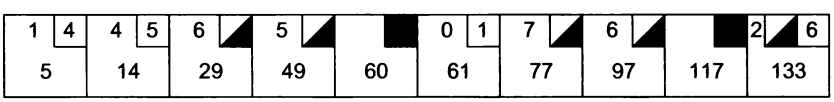
A spare frame is scored by adding ten, plus the number of pins knocked down by the next ball, to the score of the previous frame.

Otherwise, a frame is scored by adding the number of pins knocked down by the two balls in the frame to the score of the previous frame.

If a strike is thrown in the tenth frame, then the player may throw two more balls to complete the score of the strike.

Likewise, if a spare is thrown in the tenth frame, the player may throw one more ball to complete the score of the spare.

Thus, the tenth frame may have three balls instead of two.



The score card above shows a typical, if rather poor, game.

In the first frame, the player knocked down 1 pin with his first ball and four more with his second. Thus, his score for the frame is a five.

In the second frame, the player knocked down four pins with his first ball and five more with his second. That makes nine pins total, added to the previous frame makes fourteen.

In the third frame, the player knocked down six pins with his first ball and knocked down the rest with his second for a spare. No score can be calculated for this frame until the next ball is rolled.

In the fourth frame, the player knocked down five pins with his first ball. This lets us complete the scoring of the spare in frame three. The score for frame three is ten, plus the score in frame two (14), plus the first ball of frame four (5), or 29. The final ball of frame four is a spare.

Frame five is a strike. This lets us finish the score of frame four which is 29 + 10 + 10 = 49.

Frame six is dismal. The first ball went in the gutter and failed to knock down any pins. The second ball knocked down only one pin. The score for the strike in frame five is 49 + 10 + 0 + 1 = 60.

The rest you can probably figure out for yourself.

SECTION 2

Agile Design

If *agility* is about building software in tiny increments, how can you ever *design* the software? How can you take the time to ensure that the software has a good structure that is flexible, maintainable, and reusable? If you build in tiny increments, aren't you really setting the stage for lots of scrap and rework in the name of refactoring? Aren't you going to miss the big picture?

In an agile team, the big picture evolves along with the software. With each iteration, the team improves the design of the system so that it is as good as it can be for the system as it is *now*. The team does not spend very much time looking ahead to future requirements and needs. Nor do they try to build in today the infrastructure to support the features they think they'll need tomorrow. Rather, they focus on the *current* structure of the system, making it as good as it can be.

Symptoms of Poor Design

How do we know if the design of the software is good? The first chapter in this section enumerates and describes symptoms of poor design. The chapter demonstrates how those symptoms accumulate in a software project and describes how to avoid them.

The symptoms are defined as follows:

1. Rigidity—The design is hard to change.
2. Fragility—The design is easy to break.
3. Immobility—The design is hard to reuse.
4. Viscosity—It is hard to do the right thing.
5. Needless Complexity—Overdesign.
6. Needless Repetition—Mouse abuse.
7. Opacity—Disorganized expression.

These symptoms are similar in nature to code smells,[1] but they are at a higher level. They are smells that pervade the overall structure of the software rather than a small section of code.

1. [Fowler99].

Principles

The rest of the chapters in this section describe principles of object-oriented design that help developers eliminate design smells and build the best designs for the current set of features.

The principles are as follows:

1. SRP—The Single Responsibility Principle
2. OCP—The Open–Closed Principle.
3. LSP—The Liskov Substitution Principle.
4. DIP—The Dependency Inversion Principle.
5. ISP—The Interface Segregation Principle.

These principles are the hard-won product of decades of experience in software engineering. They are not the product of a single mind, but they represent the integration of the thoughts and writings of a large number of software developers and researchers. Although they are presented here as principles of object-oriented design, they are really special cases of long-standing principles of software engineering.

Smells and Principles

A design smell is a symptom, it's something that can be measured, subjectively if not objectively. Often, the smell is caused by the violation of one or more of the principles. For example, the smell of Rigidity is often a result of insufficient attention to The Open–Closed Principle (OCP).

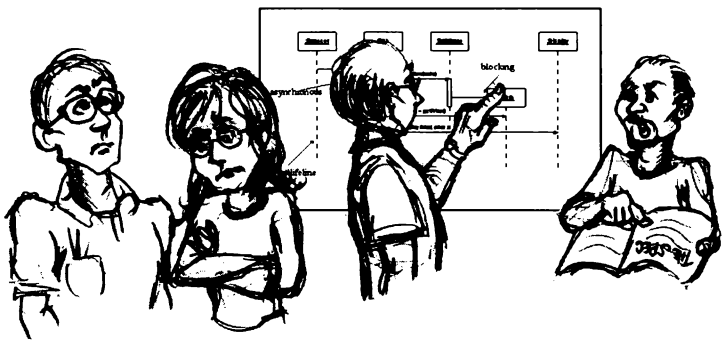
Agile teams apply principles to remove smells. They don't apply principles when there are no smells. It is a mistake to unconditionally conform to a principle just because it is a principle. Principles are not a perfume to be liberally scattered all over the system. Overconformance to the principles leads to the design smell of Needless Complexity.

Bibliography

1. Martin, Fowler. *Refactoring*. Addison–Wesley. 1999.

7

What Is Agile Design?



© Jennifer M. Kohnke

"After reviewing the software development life cycle as I understood it, I concluded that the only software documentation that actually seems to satisfy the criteria of an engineering design is the source code listings."

—Jack Reeves

In 1992, Jack Reeves wrote a seminal article in the *C++ Journal* entitled "What is Software Design?"[1] In this article, Reeves argues that the design of a software system is documented primarily by its source code. The diagrams representing the source code are ancillary to the design and are not the design itself. As it turns out, Jack's article was a harbinger of agile development.

In the pages that follow, we will often talk about "The Design." You should not take that to mean a set of UML diagrams separate from the code. A set of UML diagrams may represent parts of a design, but it is not *the* design. The design of a software project is an abstract concept. It has to do with the overall shape and structure of the program as well as the detailed shape and structure of each module, class, and method. It can be represented by many different media, but its final embodiment is source code. In the end, the source code is the design.

What Goes Wrong with Software?

If you are lucky, you start a project with a clear picture of what you want the system to be. The design of the system is a vital image in your mind. If you are luckier still, the clarity of that design makes it to the first release.

Then, something goes wrong. The software starts to rot like a piece of bad meat. As time goes by, the rot spreads and grows. Ugly festering sores and boils accumulate in the code, making it harder and harder to maintain.

1. [Reeves92] This is a great paper. I strongly recommend that you read it. I have included it in this book in Appendix D.

Eventually, the sheer effort required to make even the simplest of changes becomes so onerous that the developers and front-line managers cry for a redesign.

Such redesigns rarely succeed. Though the designers start out with good intentions, they find that they are shooting at a moving target. The old system continues to evolve and change, and the new design must keep up. The warts and ulcers accumulate in the new design before it ever makes it to its first release.

Design Smells—The Odors of Rotting Software

You know that the software is rotting when it starts to exhibit any of the following odors:

1. Rigidity—The system is hard to change because every change forces many other changes to other parts of the system.
2. Fragility—Changes cause the system to break in places that have no conceptual relationship to the part that was changed.
3. Immobility—It is hard to disentangle the system into components that can be reused in other systems.
4. Viscosity—Doing things right is harder than doing things wrong.
5. Needless Complexity—The design contains infrastructure that adds no direct benefit.
6. Needless Repetition—The design contains repeating structures that could be unified under a single abstraction.
7. Opacity—It is hard to read and understand. It does not express its intent well.

Rigidity. Rigidity is the tendency for software to be difficult to change, even in simple ways. A design is rigid if a single change causes a cascade of subsequent changes in dependent modules. The more modules that must be changed, the more rigid the design.

Most developers have faced this situation in one way or another. They are asked to make what appears to be a simple change. They look the change over and make a reasonable estimate of the work required. But later, as they work though the change, they find that there are repercussions to the change that they hadn't anticipated. They find themselves chasing the change through huge portions of the code, modifying far more modules than they had first estimated. In the end, the changes take far longer than the initial estimate. When asked why their estimate was so poor they repeat the traditional software developers' lament, "It was a lot more complicated than I thought!"

Fragility. Fragility is the tendency of a program to break in many places when a single change is made. Often, the new problems are in areas that have no conceptual relationship with the area that was changed. Fixing those problems leads to even more problems, and the development team begins to resemble a dog chasing its tail.

As the fragility of a module increases, the likelihood that a change will introduce unexpected problems approaches certainty. This seems absurd, but such modules are not at all uncommon. These are the modules that are constantly in need of repair—the ones that are never off the bug list, the ones that the developers know need to be redesigned (but nobody wants to face the spectre of redesigning them), the ones that get *worse* the more you fix them.

Immobility. A design is immobile when it contains parts that could be useful in other systems, but the effort and risk involved with separating those parts from the original system are too great. This is an unfortunate, but very common, occurrence.

Viscosity. Viscosity comes in two forms: viscosity of the software and viscosity of the environment.

When faced with a change, developers usually find more than one way to make that change. Some of the ways preserve the design; others do not (i.e., they are hacks.) When the design-preserving methods are harder to employ than the hacks, the viscosity of the design is high. It is easy to do the wrong thing, but hard to do the right thing. We want to design our software such that the changes that preserve the design are easy to make.

Viscosity of environment comes about when the development environment is slow and inefficient. For example, if compile times are very long, developers will be tempted to make changes that don't force large recompiles, even though those changes don't preserve the design. If the source-code control system requires hours to check in just a few files, then developers will be tempted to make changes that require as few check-ins as possible, regardless of whether the design is preserved.

In both cases, a viscous project is a project in which the design of the software is hard to preserve. We want to create systems and project environments that make it easy to preserve the design.

Needless Complexity. A design contains needless complexity when it contains elements that aren't currently useful. This frequently happens when developers anticipate changes to the requirements, and put facilities in the software to deal with those potential changes. At first, this may seem like a good thing to do. After all, preparing for future changes should keep our code flexible and prevent nightmarish changes later.

Unfortunately, the effect is often just the opposite. By preparing for too many contingencies, the design becomes littered with constructs that are never used. Some of those preparations may pay off, but many more do not. Meanwhile the design carries the weight of these unused design elements. This makes the software complex and hard to understand.

Needless Repetition. Cut and paste may be useful text-editing operations, but they can be disastrous code-editing operations. All too often, software systems are built upon dozens or hundreds of repeated code elements. It happens like this:

Ralph needs to write some code that fravles the arvadent. He looks around in other parts of the code where he suspects other arvadent fravling has occurred and finds a suitable stretch of code. He cuts and pastes that code into his module, and he makes the suitable modifications.

Unbeknownst to Ralph, the code he scraped up with his mouse was put there by Todd, who scraped it out of a module written by Lilly. Lilly was the first to fravle an arvadent, but she realized that fravling an arvadent was very similar to fravling a garnatosh. She found some code somewhere that fravled a garnatosh, cut and paste it into her module and modified it as necessary.

When the same code appears over and over again, in slightly different forms, the developers are missing an abstraction. Finding all the repetition and eliminating it with an appropriate abstraction may not be high on their priority list, but it would go a long way toward making the system easier to understand and maintain.

When there is redundant code in the system, the job of changing the system can become arduous. Bugs found in such a repeating unit have to be fixed in every repetition. However, since each repetition is slightly different from every other, the fix is not always the same.

Opacity. Opacity is the tendency of a module to be difficult to understand. Code can be written in a clear and expressive manner, or it can be written in an opaque and convoluted manner. Code that evolves over time tends to become more and more opaque with age. A constant effort to keep the code clear and expressive is required in order to keep opacity to a minimum.

When developers first write a module, the code may seem clear to them. That is because they have immersed themselves within it, and they understand it at an intimate level. Later, after the intimacy has worn off, they may return to that module and wonder how they could have written anything so awful. To prevent this, developers need to put themselves in their readers' shoes and make a concerted effort to refactor their code so that their readers can understand it. They also need to have their code reviewed by others.

What Stimulates the Software to Rot?

In nonagile environments, designs degrade because requirements change in ways that the initial design did not anticipate. Often, these changes need to be made quickly, and they may be made by developers who are not

familiar with the original design philosophy. So, though the change to the design works, it somehow violates the original design. Bit by bit, as the changes continue, these violations accumulate, and the design begins to smell.

However, we cannot blame the drifting of the requirements for the degradation of the design. We, as software developers, know full well that requirements change. Indeed, most of us realize that the requirements are the most volatile elements in the project. If our designs are failing due to the constant rain of changing requirements, it is our designs and practices that are at fault. We must somehow find a way to make our designs resilient to such changes and employ practices that protect them from rotting.

Agile Teams Don't Allow the Software to Rot

An agile team thrives on change. The team invests little up front; therefore, it is not vested in an aging initial design. Rather, they keep the design of the system as clean and simple as possible, and back it up with lots of unit tests and acceptance tests. This keeps the design flexible and easy to change. The team takes advantage of that flexibility in order to continuously improve the design so that each iteration ends with a system whose design is as appropriate as it can be for the requirements in that iteration.

The "Copy" Program

Watching a design rot may help illustrate the above points. Let's say your boss comes to you early Monday morning and asks you to write a program that copies characters from the keyboard to the printer. Doing some quick mental exercises in your head, you come to the conclusion that this will be less than ten lines of code. Design and coding time should be a lot less than one hour. What with cross-functional group meetings, quality education meetings, daily group progress meetings, and the three current crises in the field, this program ought to take you about a week to complete—if you stay after hours. However, you always multiply your estimates by three.

"Three weeks," you tell your boss. He harumphs and walks away, leaving you to your task.

The Initial Design. You have a bit of time right now before that process review meeting begins, so you decide to map out a design for the program. Using structured design you come up with the structure chart in Figure 7-1.

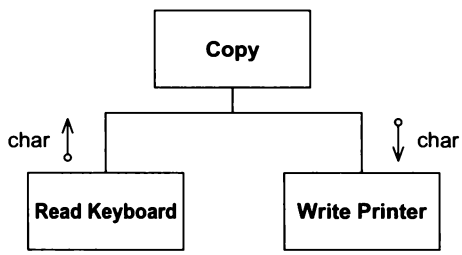


Figure 7-1 `Copy` Program Structure Chart

There are three modules, or subprograms, in the application. The `Copy` module calls the other two. The copy program fetches characters from the `Read Keyboard` module and routes them to the `Write Printer` module.

You look at your design and see that it is good. You smile and then leave your office to go to that review. At least you'll be able to get a little sleep there.

On Tuesday, you come in a bit early so that you can finish up the `Copy` program. Unfortunately, one of the crises in the field has warmed up over night, and you have to go to the lab and help debug a problem. On your lunch break, which you finally take at 3 p.m., you manage to type in the code for the `Copy` program. The result is Listing 7-1.

Listing 7-1

The Copy Program

```
void Copy()
{
  int c;
  while ((c=RdKbd()) != EOF)
    WrtPrt(c);
}
```

You just manage to save the edit, when you realize that you are already late for a quality meeting. You know this is an important one; they are going to be talking about the magnitude of zero defects. So you wolf down your Twinkies and coke and head off to the meeting.

On Wednesday, you come in early again, and this time nothing seems to be amiss. So you pull up the source code for the `Copy` program and begin to compile it. Lo and behold, it compiles first time with no errors! It's a good thing, too, because your boss calls you into an unscheduled meeting about the need to conserve laser printer toner.

On Thursday, after spending four hours on the phone with a service technician in Rocky Mount, North Carolina, walking him through the remote debugging and error logging commands in one of the more obscure components of the system, you grab a Ho Ho and then test your `Copy` program. It works, first time! Good thing, too, because your new co-op student has just erased the master source code directory from the server, and you have to go find the latest backup tapes and restore it. Of course the last full backup was taken three months ago, and you have ninety-four incremental backups to restore on top of it.

Friday, is completely unbooked. Good thing too, because it takes all day to get the `Copy` program successfully loaded into your source code control system.

Of course the program is a raging success, and gets deployed throughout your company. Your reputation as an ace programmer is once again confirmed, and you bask in the glory of your achievements. With luck, you might actually produce thirty lines of code this year!

The Requirements They Are a-Changin'. A few months later, your boss comes to you and says that sometimes they'd like the `Copy` program to be able to read from the paper tape reader. You gnash your teeth and roll your eyes. You wonder why people are always changing the requirements. Your program wasn't designed for a paper tape reader! You warn your boss that changes like these are going to destroy the elegance of your design. Nevertheless, your boss is adamant. He says the users really need to read characters from the paper tape reader from time to time.

So, you sigh and plan your modifications. You'd like to add a boolean argument to the `Copy` function. If true, then you'd read from the paper tape reader; if false, you'd read from the keyboard as before. Unfortunately, there are so many other programs using the `Copy` program now, that you can't change the interface. Changing the interface would cause weeks and weeks of recompiling and retesting. The system test engineers alone would lynch you, not to mention the seven guys in the configuration control group. And the process police would have a field day forcing all kinds of code reviews for every module that called `Copy`!

No, changing the interface is out. But then, how can you let the `Copy` program know that it must read from the paper tape reader? You'll use a global of course! You'll also use the best and most useful feature of the C suite of languages, the `?:` operator! Listing 7-2 shows the result.

Listing 7-2

First modification of Copy program

```
bool ptFlag = false;
// remember to reset this flag
void Copy()
```

```
{
  int c;
  while ((c=(ptflag ? RdPt() : RdKbd())) != EOF)
    WrtPrt(c);
}
```

Callers of `Copy` who want to read from the paper tape reader must first set the `ptFlag` to true. Then they can call `Copy`, and it will happily read from the paper tape reader. Once `Copy` returns, the caller must reset the `ptFlag`, otherwise the next caller may mistakenly read from the paper tape reader rather than the keyboard. To remind the programmers of their duty to reset this flag, you have added an appropriate comment.

Once again, you release your software to critical acclaim. It is even more successful than before, and hordes of eager programmers are waiting for an opportunity to use it. Life is good.

Give 'em an inch... Some weeks later, your boss (who is still your boss despite three corporate-wide reorganizations in as many months) tells you that the customers would sometimes like the `Copy` program to output to the paper tape punch.

Customers! They are always ruining your designs. *Writing software would be a lot easier if it weren't for customers.*

You tell your boss that these incessant changes are having a profoundly negative effect upon the elegance of your design. You warn him that if changes continue at this horrid pace, the software will be impossible to maintain before year end. Your boss nods knowingly, and then tells you to make the change anyway.

This design change is similar to the one before it. All we need is another global and another `?:` operator! Listing 7-3 shows the result of your endeavors.

Listing 7-3

```
bool ptFlag = false;
bool punchFlag = false;
// remember to reset these flags
void Copy()
{
  int c;
  while ((c=(ptflag ? RdPt() : RdKbd())) != EOF)
    punchFlag ? WrtPunch(c) : WrtPrt(c);
}
```

You are especially proud of the fact that you remembered to change the comment. Still, you worry that the structure of your program is beginning to topple. Any more changes to the input device will certainly force you to completely restructure the `while`-loop conditional. Perhaps it's time to dust off your resume...

Expect Changes. I'll leave it to you to determine just how much of the above was satirical exaggeration. The point of the story was to show how the design of a program can rapidly degrade in the presence of change. The original design of the `Copy` program was simple and elegant. Yet after only two changes, it has begun to show the signs of Rigidity, Fragility, Immobility, Complexity, Redundancy, and Opacity. This trend is certainly going to continue, and the program will become a mess.

We might sit back and blame this on the changes. We might complain that the program was well designed for the original spec, and that the subsequent changes to the spec caused the design to degrade. However, this ignores one of the most prominent facts in software development: *requirements always change*!

Remember, the most volatile things in most software projects are the requirements. The requirements are continuously in a state of flux. This is a fact that we, as developers, must accept! *We live in a world of changing*

requirements, and our job is to make sure that our software can survive those changes. If the design of our software degrades because the requirements have changed, then we are not being agile.

Agile Design of the `Copy` Example

An agile development might begin exactly the same way with the code in Listing 7-1.[2] When the boss asked the agile developers to make the program read from the paper tape reader, they would have responded by changing the design to be resilient to that kind of change. The result might have been something like Listing 7-4.

Listing 7-4

Agile version 2 of Copy

```
class Reader
{
  public:
    virtual int read() = 0;
};

class KeyboardReader : public Reader
{
  public:
    virtual int read() {return RdKbd();}
};

KeyboardReader GdefaultReader;

void Copy(Reader& reader = GdefaultReader)
{
  int c;
  while ((c=reader.read()) != EOF)
    WrtPrt(c);
}
```

Instead of trying to patch the design to make the new requirement work, the team siezes the opportunity to improve the design so that it will be resilient to that kind of change in the future. From now on, whenever the boss asks for a new kind of input device, the team will be able to respond in a way that does not cause degradation to the `Copy` program.

The team has followed the *Open–Closed Principle (OCP)*, which we will be reading about in Chapter 9. This principle directs us to design our modules so that they can be extended without modification. That's exactly what the team has done. Every new input device that the boss asks for can be provided without modifying the `Copy` program.

Note, however, that the team did not try to anticipate how the program was going to change when they first designed the module. Instead, they wrote it in the simplest way they could. It was only when the requirements did eventually change, that they changed the design of the module to be resilient to that kind of change.

One could argue that they only did half the job. While they were protecting themselves from different input devices, they could also have protected themselves from different output devices. However, the team really has no idea if the output devices will ever change. To add the extra protection now would be work that served no current puprose. It's clear that if such protection is needed, it will be easy to add later. So, there's really no reason to add it now.

2. Actually the practice of test-driven development would very likely force the design to be flexible enough to endure the boss without change. However, in this example, we'll ignore that.

How Did the Agile Developers Know What to Do?

The agile developers in the example above built an abstract class to protect them from changes to the input device. How did they know how to do that? This has to do with one of the fundamental tenets of object-oriented design.

The initial design of the `Copy` program is inflexible because of the *direction* of its dependencies. Look again at Figure 7-1. Notice that the `Copy` module depends directly on the `KeyboardReader` and the `PrinterWriter`. The `Copy` module is a high-level module in this application. It sets the policy of the application. It knows how to copy characters. Unfortunately, it has also been made dependent on the low-level details of the keyboard and printer. Thus, when the low-level details change, the high-level policy is affected.

Once the inflexibility was exposed, the agile developers knew that the dependency from the `Copy` module to the input device needed to be *inverted*[3] so that `Copy` would no longer depend on the input device. They then employed the STRATEGY[4] pattern to create the desired inversion.

So, in short, the agile developers knew what to do because

1. They detected the problem by following agile practices;
2. They diagnosed the problem by applying design principles; and
3. They solved the problem by applying the appropriate design pattern.

The interplay between these three aspects of software development *is* the act of design.

Keeping the Design As Good As It Can Be

Agile developers are dedicated to keeping the design as appropriate and clean as possible. This is not a haphazard or tentative commitment. Agile developers do not "clean up" the design every few weeks. Rather, they keep the software as clean, simple, and expressive as they possibly can, every day, every hour, and even every minute. They never say, "We'll go back and fix that later." They never let the rot begin.

The attitude that agile developers have toward the design of the software is the same attitude that surgeons have toward sterile procedure. Sterile procedure is what makes surgery *possible*. Without it, the risk of infection would be far too high to tolerate. Agile developers feel the same way about their designs. The risk of letting even the tiniest bit of rot begin is too high to tolerate.

The design must remain clean, and since the source code is the most important expression of the design, it must remain clean, too. Professionalism dictates that we, as software developers, cannot tolerate code rot.

Conclusion

So, what is agile design? Agile design is a process, not an event. It's the continous application of principles, patterns, and practices to improve the structure and readability of the software. It is the dedication to keeping the design of the system as simple, clean, and expressive as possible at all times.

During the chapters that follow, we'll be investigating the principles and patterns of software design. As you read them, remember that an agile developer does not apply those principles and patterns to a big, up-front design. Rather, they are applied from iteration to iteration in an attempt to keep the code, and the design it embodies, clean.

Bibliography

1. Reeves, Jack. What Is Software Design? *C++ Journal*, Vol. 2, No. 2. 1992. Available at `http://www.bleading-edge.com/Publications/C++Journal/Cpjour2.htm`.

3. See *The Dependency-Inversion Principle (DIP)* in Chapter 11.
4. We'll learn about STRATEGY in Chapter 14.

8

SRP: The Single-Responsibility Principle



None but Buddha himself must take the responsibility of giving out occult secrets...

—E. Cobham Brewer 1810–1897.
Dictionary of Phrase and Fable. 1898.

This principle was described in the work of Tom DeMarco[1] and Meilir Page-Jones.[2] They called it *cohesion*. They defined cohesion as the functional relatedness of the elements of a module. In this chapter we'll shift that meaning a bit and relate cohesion to the forces that cause a module, or a class, to change.

SRP: The Single-Responsibility Principle

A class should have only one reason to change.

Consider the bowling game from Chapter 6. For most of its development, the `Game` class was handling two separate responsibilities. It was keeping track of the current frame, and it was calculating the score. In the end, RCM and RSK separated these two responsibilities into two classes. The `Game` kept the responsibility to keep track of frames, and the `Scorer` got the responsibility to calculate the score. (See page 78.)

1. [DeMarco79], p. 310.
2. [Page-Jones88], Chapter 6, p. 82.